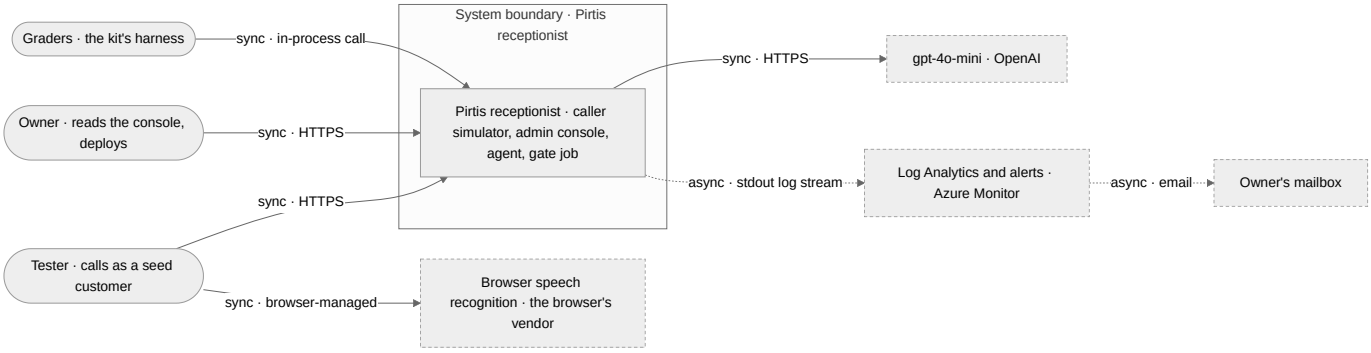
> Verified against `3e57ec9` on 2026-10-31.

Containers

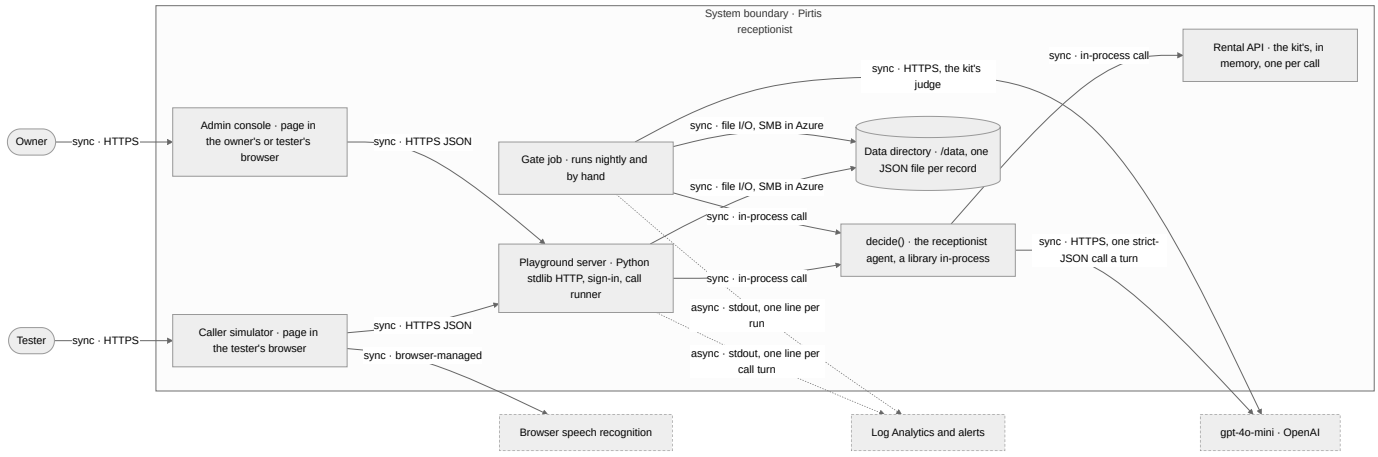
What runs, and what we don't control. Level 1 is the system as one box; level 2 opens it. Where it runs (resource group, registry, identity) is in `deployment.md`; who does what in order is in `sequences.md`; where personal data goes is in `data-flow.md`.

Level 1 · context



The graders don't use the deployment: they run the kit's harness, which calls `decide()` in their own process. Everything else is the deployed system.

Level 2 · containers



Container	What it does	Evidence
Caller simulator	Pick a seed customer or an unknown number; type, or tap (or hold) to talk; the fogged booking card condenses as facts are heard; thumbs on every reply; scenario cards and the rating form	`python/pirtis/playground/static/caller.html`, `python/pirtis/playground/static/js/caller.js`
Admin console	Reliability (gate runs, the thermal view), Conversations, Traces, Feedback, Fleet, Human eval; all computed from content-free records except a stored call's trace	`python/pirtis/playground/static/admin.html`, `python/pirtis/playground/static/js/admin.js`, `python/pirtis/playground/analytics.py`

Container	What it does	Evidence
Playground server	The JSON API and the two pages; the sign-in (accounts we create, signed sessions); runs each call turn through `decide()` and the kit's handlers, saves the call after every turn	`python/pirtis/playground/server.py`, `python/pirtis/playground/auth.py`, `python/pirtis/playground/calls.py`
`decide()`	Turn gate (no model call), one strict-JSON understanding call, a deterministic policy that owns every rule and write, code-rendered speech, one telemetry line	`python/pirtis/agent/agent.py`, `python/pirtis/agent/turn.py`, `python/pirtis/agent/understand.py`, `python/pirtis/agent/policy.py`, `python/pirtis/agent/rules.py`, `python/pirtis/agent/speech.py`
Rental API	The kit's fixed API over an in-memory store, seeded per call from the fixtures' seed: units, availability, customers, bookings	`python/pirtis/api/memory.py`, `fixtures/seeds/base.json`
Gate job	The frozen gate in one process: the kit's Runner over the nightly suite, the extractor eval, a saved gate run, one summary line, pruning	`python/pirtis/ops/gate_job.py`
Data directory	Sessions, feedback, ratings, turn records, gate runs, calibrations; atomic writes, retention per kind	`python/pirtis/ops/store.py`, `python/pirtis/ops/model.py`

Outside the boundary

Dependency	Vendor · region	Edge	If it is down	Fallback	Evidence
gpt-4o-mini	OpenAI API · [CONFIRM: processing region, US unless a data-residency project is set]. Azure OpenAI in an EU region instead via `LLM_BASE_URL`	sync · HTTPS	a turn that needs the model is put through to a person; an unreadable answer gets "could you say that again?"; the gate job fails with `llm_errors` above zero	turns the turn gate settles (still talking, silence, a request for a person) need no model; none for the rest	`python/pirtis/llm/client.py`, `python/pirtis/agent/agent.py`
Browser speech recognition	the browser's vendor (Chrome sends audio to Google's service; [CONFIRM: Safari and Edge]) · [CONFIRM: region]	sync · browser-managed	holding to talk does nothing useful	typing, always available	`python/pirtis/playground/static/js/caller.js`
Azure Files, hosting the data directory	Microsoft Azure · Germany West Central	sync · SMB	calls can't be saved or resumed; a gate run isn't saved	none	`deploy/terraform/main.tf`
Log Analytics, alerts, email	Microsoft Azure Monitor · Germany West Central	async · container stdout	no log queries and no alerts; calls go on	the admin console reads the data directory, not the logs	`deploy/terraform/monitoring.tf`
Container registry	Azure Container Registry · Germany West Central	sync · HTTPS image pull	a new revision or a job run can't start; the running app is unaffected	none	`deploy/terraform/main.tf`

Not built:

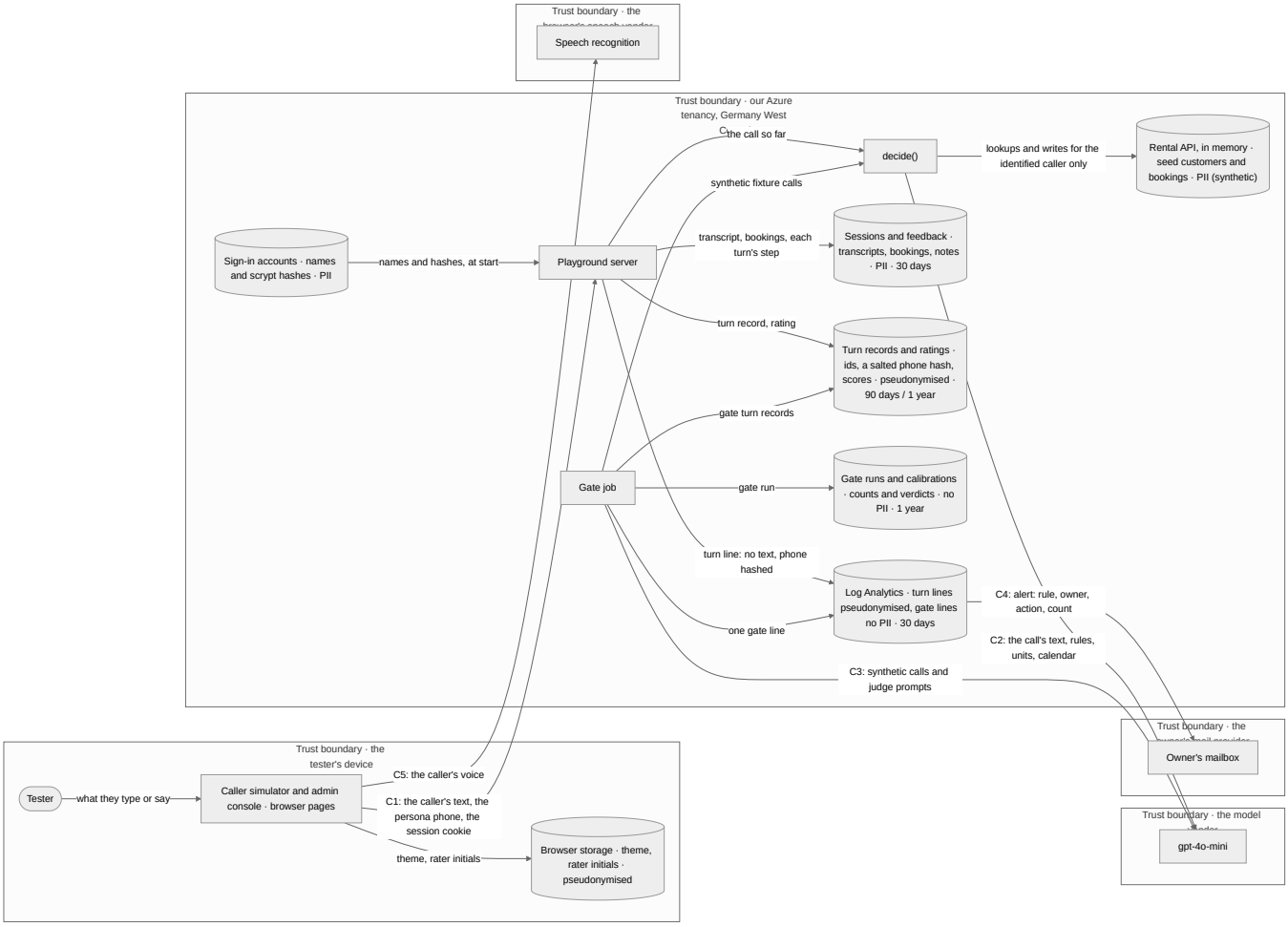
- real telephony; the caller simulator stands in for the phone line

- streaming speech; a turn is one request
- a queue; every call turn is synchronous
- single sign-on; accounts are made by hand

> Verified against `3e57ec9` on 2026-10-31.

Data flow

Where personal data sits, and what happens to it just before it leaves. The customers are made up (the fixtures' seed), and testers call as those customers. A tester's own words and voice are still personal data, so they are labelled as such.



Stores

Store	Label	Kept	Evidence
Sessions	PII: what the tester typed or said, the bookings the call made, each turn's step	30 days	`python/pirtis/ops/model.py`, `python/pirtis/playground/calls.py`
Feedback	PII: a transcript snapshot and the rater's note	30 days	`python/pirtis/ops/model.py`
Turn records	pseudonymised: ids, a salted hash of the phone's digits, counts; no text	90 days	`python/pirtis/agent/telemetry.py`, `python/pirtis/ops/model.py`
Human ratings	pseudonymised: the rater's initials, a session id, scores	1 year	`python/pirtis/ops/model.py`
Gate runs, calibrations	no PII: fixture ids, verdicts, counts	1 year	`python/pirtis/ops/model.py`
Sign-in accounts	PII: account names and scrypt hashes, a Container App secret	until removed	`python/pirtis/playground/auth.py`, `deploy/Makefile`
Rental API	PII in shape, synthetic in content: the seed's customers, phones and bookings; rebuilt per call, never saved	the call	`python/pirtis/api/memory.py`, `fixtures/seeds/base.json`

Store	Label	Kept	Evidence
Log Analytics	pseudonymised: turn lines (phone hash, ids, no text); gate lines and access lines (method, path without its query, status) hold no personal data	30 days	`python/pirtis/agent/telemetry.py`, `python/pirtis/playground/server.py`, `deploy/terraform/main.tf`
Browser storage	pseudonymised: the theme and the rater's initials, on the tester's own device	until cleared	`python/pirtis/playground/static/js/caller.js`

Retention is applied by the gate job's prune on every run (`python/pirtis/ops/store.py`).

Boundary crossings

ID	From → To	Data	Before it crosses	Region	Transfer mechanism (GDPR Ch. V)	Retention	Evidence
C1	caller simulator → playground server (and the replies back)	the caller's text (speech already turned to text in the browser), the persona's phone, the session cookie; back: our replies, the booking card, a stored call's trace for the console	TLS only (HTTPS, HSTS); a signed-in session is required; spending is capped per account and per day	Germany West Central (EU)	none needed (the receiving side is in the EEA)	sessions 30 days	`python/pirtis/playground/server.py`, `python/pirtis/playground/auth.py`
C2	`decide()` → model vendor	the call so far as text: caller lines, our lines, short notes ("caller identified", the caller's own bookings by sauna and dates); the rules; the units; a 42-day calendar	nothing is redacted: the caller's words go as said, a name included if they spoke it. Never sent: the phone number, other customers' data, past bookings' site and car details	[CONFIRM: OpenAI's processing region; the US unless a data-residency project is configured]	[CONFIRM: OpenAI's DPA with SCCs, Art. 46(2)(c)]; none needed on Azure OpenAI in an EU region	[CONFIRM: OpenAI API retention; abuse monitoring up to 30 days unless zero retention is agreed]	`python/pirtis/agent/understand.py`, `python/pirtis/llm/client.py`

ID	From → To	Data	Before it crosses	Region	Transfer mechanism (GDPR Ch. V)	Retention	Evidence
C3	gate job → model vendor	synthetic fixture calls, and the kit's judge prompts with our replies	nothing: synthetic data only	as C2	as C2	as C2	`python/pirtis/ops/gate_job.py`, `python/pirtis/harness/judge.py`
C4	Azure Monitor → owner's mailbox	an alert: the rule's name, its owner-and-action text, a count, a link; the queries project only versions, case ids and counts	nothing: the rules read content-free lines	[CONFIRM: the mail provider's region]	[CONFIRM: depends on the mail provider]	[CONFIRM: the mailbox's retention]	`deploy/terraform/monitoring.tf`
C5	caller simulator → the browser's speech vendor	the tester's voice, while they hold the mic button	nothing: the browser sends the audio itself (Chrome to Google's service)	[CONFIRM: the browser vendor's region]	[CONFIRM: the browser vendor's terms]	[CONFIRM: the browser vendor's retention]	`python/pirtis/playground/static/js/caller.js`

What this diagram says plainly:

- **C2 leaves the EEA on the OpenAI path.** The data is synthetic customers plus whatever a tester types, which is why testers call as seed personas and never as themselves. Pointing `LLM_BASE_URL` at Azure OpenAI in an EU region keeps C2 inside the EEA.
- **C5 is the browser's doing,** not ours, and a tester avoids it by typing.

Privacy threats (a LINDDUN pass over this diagram)

Threat	Where	Finding	Handled by
Linking	turn records	the salted phone hash links one caller's turns across calls, by design for analytics	the salt is a secret; records expire after 90 days
Identifying	sessions, feedback	a tester can say their own name or details	seed personas only (scenario cards, the dial page's note); 30-day expiry
Data disclosure	C2, C5	call text leaves the EEA to the model vendor; voice goes to the browser's vendor	synthetic personas; typing as an alternative; the EU model path. Recorded as a known limit (SPEC §12)
Unawareness and unintervenability	the caller simulator	the only notice is the dial page's line that calls are kept 30 days; a tester can't delete their own calls	the owner deletes on request (files on the share); a self-service delete is not built
Non-compliance	C2	no DPA with the model vendor is recorded here	[CONFIRM: OpenAI DPA]

Not built:

- redaction before C2 (names spoken by the caller go to the model as said)

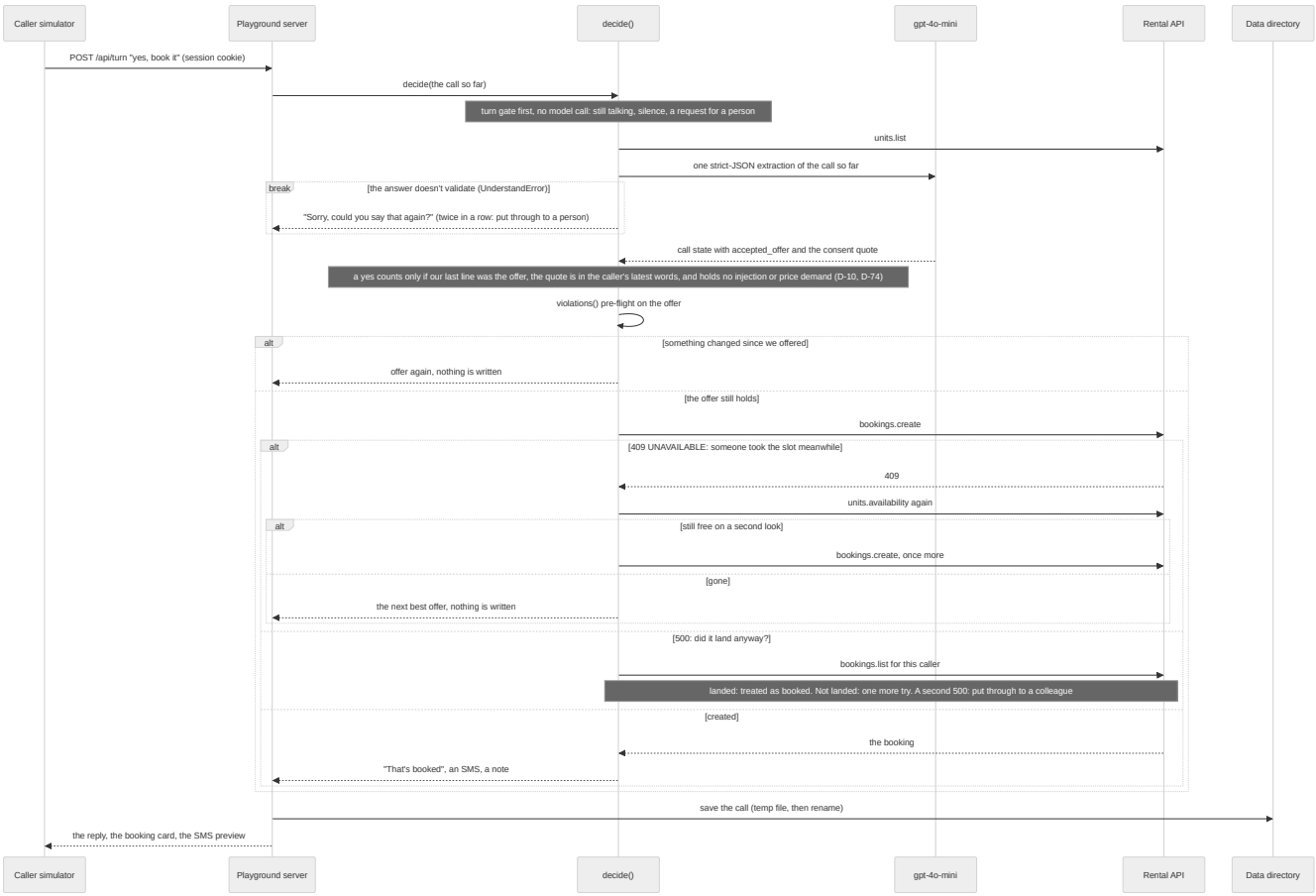
- a self-service way for a tester to delete their calls
- a privacy notice beyond the dial page's line

> Verified against `3e57ec9` on 2026-10-31.

Sequences

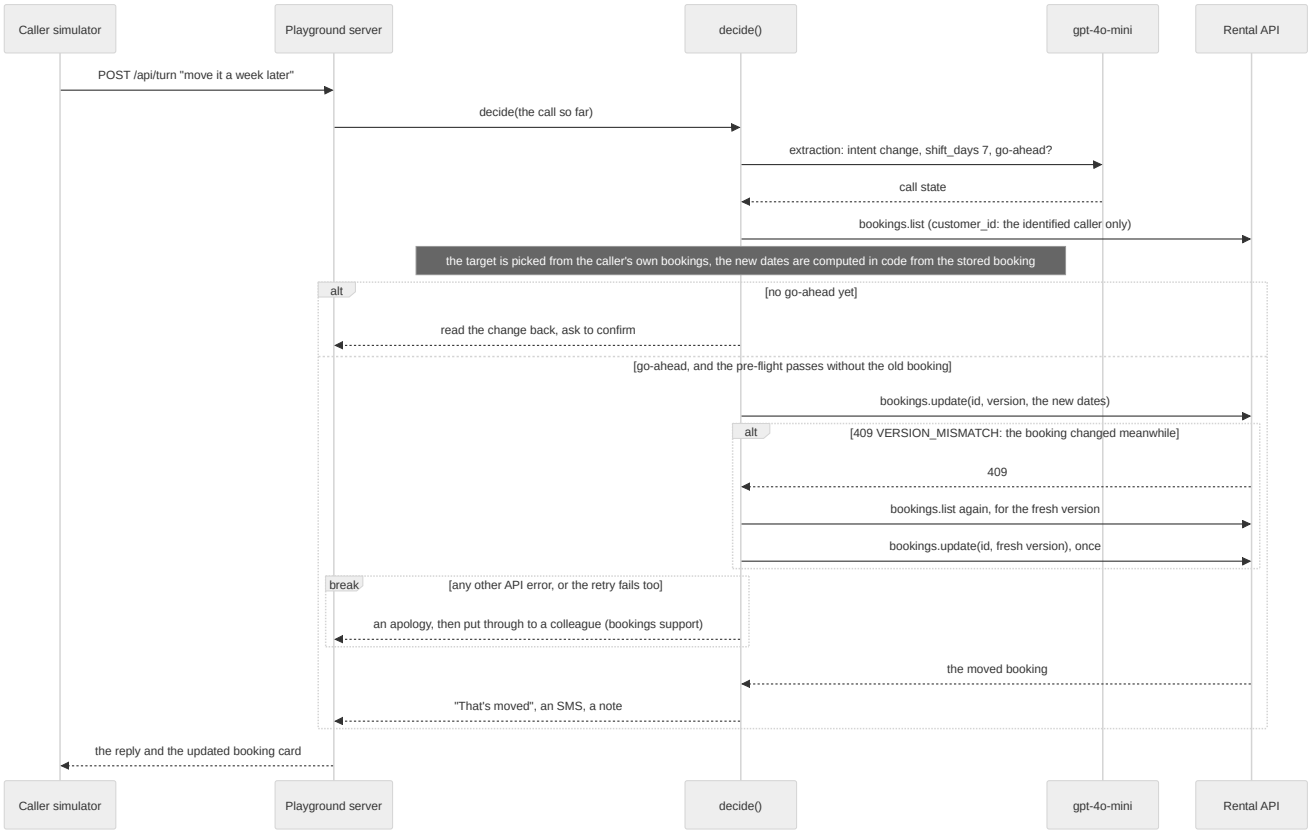
The two paths that write something a customer relies on: a new booking, and a change to one. The participants are the containers in `containers.md`.

1 · A booking, on the caller's yes to our read-back



The pre-flight `violations()` mirrors every rule the API enforces and is property-tested against it (`python/pirtis/agent/rules.py`), so a 422 can't happen by construction. The retries are in `create()` (`python/pirtis/agent/policy.py`); a failure no branch handles becomes a hand-off to a person (`python/pirtis/agent/agent.py`).

2 · Moving or cancelling a booking



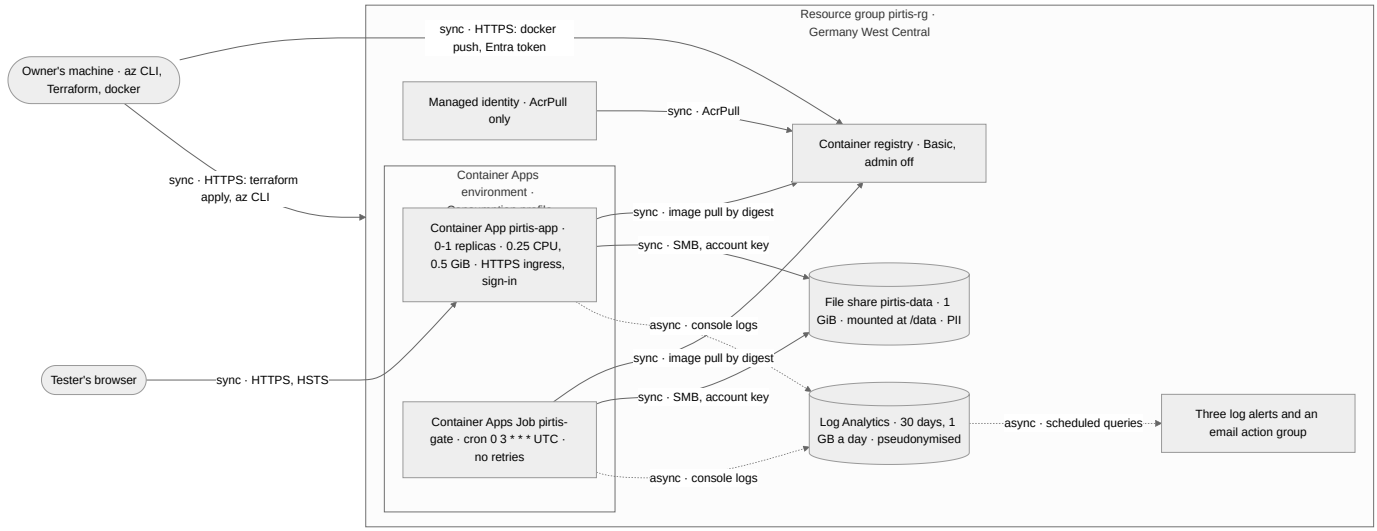
A cancel is the same path with `status: cancelled` as the change. The retry is `update_with_retry()` in `python/pirtis/agent/policy.py`.

Not built: an idempotency key on writes. A double submit is safe today only because one call is one session handled one turn at a time, and the 500 branch looks before retrying.

> Verified against `3e57ec9` on 2026-10-31.

Deployment

Where the containers run. This view is about infrastructure; what talks to what is in `containers.md`. Commands, costs and the bootstrap order are in `deploy/README.md`.



Resource	Settings that matter	Evidence
Container App `pirtis-app`	0 to 1 replicas; 0.25 CPU, 0.5 GiB; external ingress, HTTPS only; probes on `/api/health`; image by digest; secrets: model key, hash salt, session key, accounts	`deploy/terraform/main.tf`
Container Apps Job `pirtis-gate`	cron `0 3 * * *` UTC; 30 min timeout; no retries (a failed gate is read, never re-run); `gate-job --suites nightly`	`deploy/terraform/main.tf`
Container registry	Basic; admin account off; pulled only by the managed identity	`deploy/terraform/main.tf`
Managed identity	one role: AcrPull on the registry	`deploy/terraform/main.tf`
Storage account and share	LRS; TLS 1.2; HTTPS only; no public blob access; one 1 GiB share, linked to the environment with the account key	`deploy/terraform/main.tf`, `deploy/Makefile`
Log Analytics	30 days; a 1 GB daily cap	`deploy/terraform/main.tf`
Alerts	gate regression, agent errors, slow turns; every 15 minutes over the last hour; email to the owner	`deploy/terraform/monitoring.tf`
The image	two stages, bases pinned by digest; uid 10001, read-only code; refuses to start without sign-in accounts	`deploy/Dockerfile`, `.dockerignore`

Secrets are set by the az CLI, never written in these files: the model key, the hash salt, the session key, the sign-in accounts, and the storage key for the share's link. Terraform's state holds the storage keys and, once refreshed, the app's secrets. So the state stays on the owner's machine, owner-only and out of git (`deploy/Makefile`).

Not built:

- remote, encrypted Terraform state
- Key Vault references for the secrets
- a managed-identity mount for the share (in preview; the account key is used today)
- private networking: the app has a public endpoint, behind its sign-in
- CI that builds, scans and deploys on every push